# COMPILER DESIGN PROJECT REPORT

*VI SEMESTER PROJECT REPORT*
*Submitted*
*to*

**MANIPAL ACADEMY OF HIGHER EDUCATION**

**School of Computer Engineering**
**COMPUTER SCIENCE ENGINEERING**

*Submitted by*


**AMEY RAJ JAIN**
CSE-B
Reg no- 235805470

**JAY DENISH CHOTHIYAWALA**
CSE-B
Reg no- 235805490

**DHYEY TANDEL**
CSE-B
Reg no- 235805422


*Under the guidance of*


**Dr. Anagha P**
Assistant Professor- Senior Scale
School of Computer Engineering
Manipal Institute of Technology, Bengaluru


MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
*(A constituent unit of MAHE, Manipal)*

| Table of Contents | Page No. |
|---|---|
| Abstract | 1 |
| 1. Introduction | 1 |
| 2. Problem Statement | 2 |
| 3. Scope and Objectives | 2 |
| 4. System Architecture | 3 |
| 5. Language Design and Grammar Overview | 3 |
| 6. Module-Wise Implementation Details | 4 |
| 7. Input/Output Workflow | 7 |
| 8. Error Handling Strategy | 11 |
| 9. Testing and Validation | 12 |
| 10. Strengths, Limitations, and Improvements | 12 |
| 11. Educational Relevance | 13 |
| 12.Conclusion | 14 |
| References | 14 |

# Abstract

**This report presents the design, implementation, and evaluation of a mini compiler built in Java for a small imperative programming language. The project demonstrates the key phases of compilation: lexical analysis, syntax analysis (parsing), intermediate code generation, and final target code generation. The compiler accepts source input from in.txt, processes it through each phase, reports errors in a centralized manner, and writes generated target code to out.txt while also printing outputs to the terminal.**

**The language supports basic constructs such as variable input/output, assignment statements, arithmetic expressions, conditional statements (if-then-else), and iterative loops (while-do). The system emphasizes clarity and educational value rather than advanced optimization. A modular architecture was adopted, with separate classes for each major concern: Lexer, Parser, ASTNode, IntermediateCodeGenerator, CodeGenerator, CompilerErrorHandler, and Main.**

**Testing confirms that the compiler correctly translates valid programs and provides robust diagnostics for invalid ones. The result is a complete, end-to-end prototype suitable for compiler design learning, demonstrations, and further academic extension.**


# 1. Introduction

**Compilers are foundational tools in computer science, translating human-readable source programs into lower-level instructions executable by machines or virtual environments. Building even a small compiler provides practical understanding of programming language theory, formal grammars, automata, parsing, syntax-directed translation, and code generation.**

**This project implements a compact compiler pipeline for a toy language. Though intentionally limited in scope, the implementation mirrors real compiler stages and design patterns. The primary goals were:**

1. **Build a functioning compiler pipeline from source input to target output.**
2. **Represent syntax through an Abstract Syntax Tree (AST).**
3. **Generate intermediate representation and final pseudo-assembly-like code.**
4. **Provide understandable error handling and reporting.**
5. **Keep architecture modular and maintainable.**

**The final system demonstrates how high-level language constructs can be broken down systematically into lower-level instructions. This makes it ideal for academic assignments and as a base for future enhancements such as semantic analysis, symbol tables, type checking, optimization, and backend expansion.**

# 2. Problem Statement

**The objective is to design and implement a compiler for a small custom language that:**

- **Reads source code from an external file.**
- **Tokenizes the source into lexical units.**
- **Parses tokens according to grammar rules.**
- **Builds an AST to represent program structure.**
- **Generates intermediate code (three-address style).**
- **Generates final target code (pseudo assembly).**
- **Handles and reports lexical and syntactic errors clearly.**
- **Writes final generated code into an output file.**

**The project should be simple enough for educational use but complete enough to demonstrate full compilation flow.**

# 3. Scope and Objectives

## 3.1 Scope

**The implemented language includes:**

- **read(identifier);**
- **write(expression);**
- **identifier := expression;**
- **if condition then statement else statement**
- **while condition do statement**
- **Arithmetic operators: +, -, *, /**
- **Relational operators: >, <, ==**
- **Parenthesized expressions**

## 3.2 Objectives

1. **Implement a deterministic lexical analyser.**
2. **Implement a recursive-descent parser.**
3. **Build hierarchical AST nodes.**
4. **Generate meaningful intermediate code with temporaries and labels.**

5. **Generate low-level final code with registers and jumps.**
6. **Centralize error management in a dedicated handler.**
7. **Ensure usability through terminal output and file output (out.txt).**

# 4. System Architecture

**The compiler follows a staged architecture:**

1. **Input Manager (Main)**
   **Loads source program from in.txt (or custom file path), handles UTF-8 BOM, initializes pipeline.**
2. **Lexical Analyzer (Lexer)**
   **Scans source characters and emits tokens such as identifiers, numbers, keywords, operators, and punctuation.**
3. **Syntax Analyzer (Parser)**
   **Consumes token stream, validates grammar, and constructs an AST.**
4. **Intermediate Code Generator (IntermediateCodeGenerator)**
   **Traverses AST and emits three-address-like intermediate instructions (t0, t1, labels).**
5. **Final Code Generator (CodeGenerator)**
   **Produces target pseudo assembly (IN, OUT, MOV, ADD, CMP, SETxx, JZ, JMP) and writes to out.txt.**
6. **Error Handler (CompilerErrorHandler)**
   **Centralized error formatting/reporting, maintains count and summary.**

**This separation of concerns improves readability, debugging, and extensibility.**

# 5. Language Design and Grammar Overview

**A simplified grammar (informal form) used by the parser is:**

- **program -> statement* EOF**
- **statement -> assignment | readStmt | writeStmt | ifStmt | whileStmt**
- **assignment -> ID ASSIGN expression SEMI**
- **readStmt -> READ LP ID RP SEMI**
- **writeStmt -> WRITE LP expression RP SEMI**
- **ifStmt -> IF condition THEN statement (ELSE statement)?**
- **whileStmt -> WHILE condition DO statement**

- **condition -> expression ((GT | LT | EQ) expression)?**
- **expression -> term ((PLUS | MINUS) term) ***
- **term -> factor ((MUL | DIV) factor) ***
- **factor -> LP expression RP | ID | NUM**

**The parser is recursive-descent, making grammar mapping direct and easy to maintain.**

# 6. Module-Wise Implementation Details

## 6.1 Token.java

**This class represents lexical tokens with:**

- **type (e.g., ID, NUM, IF, PLUS)**
- **value (lexeme text)**
- **line (line number for diagnostics)**

**It is used across lexer, parser, and output display.**

## 6.2 Lexer.java

**Lexer reads the source string character by character:**

- **Skips whitespace and tracks line count.**
- **Detects identifiers and keywords (if, then, else, while, do, read, write).**
- **Detects numbers (NUM).**
- **Detects operators and symbols.**
- **Handles compound operators:= and ==.**
- **Emits EOF at end.**

**Error behavior:**

- **Invalid characters are reported through CompilerErrorHandler.**
- **Invalid standalone = and : produce explicit lexical error messages.**

**Output:**

- **A list of tokens displayed as lexer output in terminal.**

## 6.3 ASTNode.java

**ASTNode models parse tree abstractions:**

- **type and value**
- **children list**
- **helper methods for adding/accessing children**

**It acts as the core IR (internal representation) between parsing and code generation.**

## 6.4 Parser.java

**Parser receives token list and uses recursive-descent methods:**

- **parseProgram, parseStatement, parseExpression, etc.**
- **expect verifies expected token types.**
- **On syntax mismatch, throws structured exceptions via CompilerErrorHandler.**

**Recovery strategy:**

- **When a statement parse fails, parser reports the error and advances token position to continue parsing later parts of the input.**
  **This allows multiple errors to be shown in one run.**

**Output:**

- **Parsed AST is printed in hierarchical form through Main.printAst.**

## 6.5 IntermediateCodeGenerator.java

**This module traverses AST and emits three-address-like IR:**

- **Creates temporary variables (t0, t1, ...).**
- **Creates labels (L0, L1, ...).**
- **Generates control-flow instructions for if and while.**
- **Handles arithmetic and relational expression translation.**

**Example:**

- **x := 5 + 3 becomes:**
  - **t0 = 5 + 3**
  - **x = t0**

## 6.6 CodeGenerator.java

**Generates final pseudo-assembly code:**

- **Input/output: IN, OUT**
- **Move: MOV**
- **Arithmetic: ADD, SUB, MUL, DIV**
- **Comparison + set flags: CMP, SETGT, SETLT, SETEQ**
- **Control flow: JZ, JMP, labels**

**Enhancements:**

- **Emits each line to terminal.**
- **Captures generated lines in a buffer.**
- **Exposes getOutput() for writing to out.txt.**

## 6.7 CompilerErrorHandler.java

**A dedicated centralized utility:**

- **Standardized error format: Error at line X: ...**
- **Reports both lexical and parser errors.**
- **Stores errors internally.**
- **Provides count and summary data.**
- **Enables end-of-run error summary.**

## 6.8 Main.java

**Orchestrates full pipeline:**

1. **Reads/creates input file.**
2. **Runs lexer and prints tokens.**
3. **Runs parser and prints AST.**
4. **Runs intermediate generation.**
5. **Runs final code generation.**
6. **Writes final code to out.txt.**
7. **Prints error summary.**

**This class improves user experience with labeled stage outputs.**

# 7. Input/Output Workflow

## 7.1 Input File (in.txt)

**Typical valid input:**

```
read(x);
x := 5 + 3;
if x > 5 then
   write(x);
else
   write(0);
while x < 10 do
   x := x + 1;
```

## 7.2 Terminal Output

**The terminal shows:**

- **Source code**

```
===== SOURCE CODE =====
read(x);
x := 5 + 3;
if x > 5 then
    write(x);
else
    write(0);
while x < 10 do
    x := x + 1;
```

- **Lexer token stream**

```
===== LEXER OUTPUT (TOKENS) =====
READ(read) at line 1
LP(() at line 1
ID(x) at line 1
RP()) at line 1
SEMI(;) at line 1
ID(x) at line 2
ASSIGN(:=) at line 2
NUM(5) at line 2
PLUS(+) at line 2
NUM(3) at line 2
SEMI(;) at line 2
IF(if) at line 3
ID(x) at line 3
GT(>) at line 3
NUM(5) at line 3
THEN(then) at line 3
WRITE(write) at line 4
LP(() at line 4
ID(x) at line 4
RP()) at line 4
SEMI(;) at line 4
ELSE(else) at line 5
WRITE(write) at line 6
LP(() at line 6
NUM(0) at line 6
RP()) at line 6
SEMI(;) at line 6
WHILE(while) at line 7
ID(x) at line 7
LT(<) at line 7
NUM(10) at line 7
DO(do) at line 7
ID(x) at line 8
ASSIGN(:=) at line 8
ID(x) at line 8
PLUS(+) at line 8
```

- **Parser AST**

```
===== PARSER OUTPUT (AST) =====
PROGRAM
  READ(x)
  ASSIGN
    ID(x)
    PLUS(+)
      NUM(5)
      NUM(3)
  IF
    GT(>)
      ID(x)
      NUM(5)
    WRITE
      ID(x)
    WRITE
      NUM(0)
  WHILE
    LT(<)
      ID(x)
      NUM(10)
    ASSIGN
      ID(x)
      PLUS(+)
        ID(x)
        NUM(1)
```

- **Intermediate code**

```
===== INTERMEDIATE CODE =====
READ x
t0 = 5 + 3
x = t0
t1 = x > 5
IF_FALSE t1 GOTO L0
PRINT x
GOTO L1
L0:
PRINT 0
L1:
L2:
t2 = x < 10
IF_FALSE t2 GOTO L3
t3 = x + 1
x = t3
GOTO L2
L3:
```

- **Final code & Error summary**

```
===== FINAL CODE =====
IN x
ADD R0, 5, 3
MOV x, R0
CMP x, 5
SETGT R1
JZ R1, L0
OUT x
JMP L1
L0:
OUT 0
L1:
L2:
CMP x, 10
SETLT R2
JZ R2, L3
ADD R3, x, 1
MOV x, R3
JMP L2
L3:

===== ERROR SUMMARY =====
No errors found.
```

## 7.3 Output File (out.txt)

**Generated final target code is stored automatically.**

**Example output:**

**IN x**

**ADD R0, 5, 3**

**MOV x, R0**

**CMP x, 5**

**SETGT R1**

**JZ R1, L0**

**OUT x**

**JMP L1**

**L0:**

**OUT 0**

**L1:**

**L2:**

**CMP x, 10**

```
SETLT R2
JZ R2, L3
ADD R3, x, 1
MOV x, R3
JMP L2
L3:
```

# 8. Error Handling Strategy

**Error handling is a key strength of the current implementation.**

## 8.1 Lexical Errors

**Detected cases:**

- **Unsupported symbols (@, etc.)**
- **Invalid partial operators (= not followed by =, : not followed by =)**

**These errors are reported with exact line numbers.**

## 8.2 Parser Errors

**Detected cases:**

- **Missing semicolon**
- **Missing parentheses**
- **Invalid statement forms**
- **Unexpected tokens in expressions/conditions**

**Parser uses controlled exception generation and local recovery to continue scanning for additional errors.**

## 8.3 Summary Reporting

**At the end of compilation:**

- **No errors found. if clean input.**
- **Total errors: N for faulty input.**

**This makes debugging faster for students and developers.**

# 9. Testing and Validation

## 9.1 Functional Testing

**The following test categories were used:**

1. **Valid sequential statements.**
2. **Nested expressions with arithmetic precedence.**
3. **Conditional statements with and without else.**
4. **Loop constructs.**
5. **Invalid tokens.**
6. **Syntax-missing cases (e.g., ;, )).**
7. **Combined erroneous multi-line programs.**

## 9.2 Expected Results

- **Valid programs generate correct tokens, AST, intermediate code, and target code.**
- **Invalid programs report multiple readable errors without immediate crash.**
- **Final code is written consistently to out.txt.**

## 9.3 Example Fault Injection

**A deliberately invalid 10-line input containing malformed operators, invalid symbols, and broken statements successfully triggered many lexical/syntax errors, confirming robustness of diagnostics.**

# 10. Strengths, Limitations, and Improvements

## 10.1 Strengths

- **End-to-end compiler pipeline implemented.**
- **Clear modular design.**
- **Easy-to-read recursive parser.**
- **AST-based translation design.**
- **Dual output support (terminal + out.txt).**
- **Centralized and improved error handling.**

## 10.2 Limitations

- **No semantic analysis (undeclared variables, type checks).**
- **No symbol table currently enforced.**
- **Single basic data domain (effectively integer-like expressions).**
- **Limited grammar (no blocks {}, no functions, no declarations).**
- **No optimization passes.**

## 10.3 Future Enhancements

1. **Add symbol table and declaration rules.**
2. **Add semantic checks before code generation.**
3. **Support multi-statement blocks for if/while.**
4. **Add boolean operators (&&, ||, !).**
5. **Add unary operators and richer expression grammar.**
6. **Generate executable bytecode/VM instructions instead of pseudo assembly.**
7. **Add optimization passes (constant folding, dead code elimination).**
8. **Export AST and IR to separate files for debugging.**
9. **Implement automated unit tests with JUnit.**
10. **Add command-line options for verbose/silent modes.**

# 11. Educational Relevance

**This project is pedagogically strong because it maps theory to implementation:**

- **Finite-state style scanning in lexer.**
- **Context-free grammar parsing through recursive descent.**
- **Tree-based intermediate representation.**
- **Control-flow translation with labels and jumps.**
- **Practical recovery-based error handling.**

**Students can understand each phase independently and still observe the full compiler pipeline as an integrated system. The codebase is compact enough for classroom discussion, lab exercises, and incremental assignments.**

# 12. Conclusion

**The mini compiler project successfully demonstrates the complete core architecture of a compiler for a simple imperative language. It reads source code from file input, performs lexical and syntax analysis, builds an AST, generates intermediate and final target code, handles errors centrally, and produces both terminal and file outputs.**

**Although simplified, the implementation captures the essence of compilation and provides a strong foundation for advanced features. The modular approach ensures maintainability and supports future extension toward semantic analysis, optimization, and richer language constructs. As an academic/compiler-design subject implementation, this project is complete, functional, and highly suitable for practical learning and demonstration.**

# References


1. **Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. *Compilers: Principles, Techniques, and Tools* (2nd ed.).**
2. **Louden, K. C. *Compiler Construction: Principles and Practice*.**
3. **Appel, A. W. *Modern Compiler Implementation* (Java/C/ML editions).**
4. **Grune, D., van Reeuwijk, K., Bal, H. E., Jacobs, C. J. H., & Langendoen, K. *Modern Compiler Design*.**
5. **Cooper, K. D., & Torczon, L. *Engineering a Compiler*.**